REQUIREMENTS

HelixKnowledge Skill Graph System — Consolidated Requirements (living doc)

Status: **requirements-gathering + research phase → P0.5 critical remediation.** Foundation (extracted Go backend) **compiles clean** (`go build ./...`=0 at HEAD `255061b`; the original "does not compile" was the pre-P0 Baseline, resolved by the build-fix commit `5532e2b` — see "Baseline", now marked SUPERSEDED). This doc is the single source of truth as scope evolves; correct it here.

Vision

A **universal**, self-growing Knowledge Skill Graph for AI CLI agents. Users request knowledge for arbitrary technologies; the system **creates skills dynamically on demand**, maps their dependency DAG, validates them (no bluff), stores them (Postgres + pgvector), and serves them to AI coding agents.

Canonical source-of-truth (original founding request) — reconciled

The operator supplied the ORIGINAL request used to prepare the MVP. It is the source of truth. No separate `.txt` attachment is present in the repo — the extracted MVP (SPEC/plan/research/project) IS the prepared start point.

CONFIRMS (already captured above): recursive skill DAG for AOSP/ Android/Java/ Kotlin/C++ + every dependency tech (R2/R13); central registrar + fully-automatic auto-growth/expansion (R2); exhaustive

reviews, zero false/faulty/bluff (R8/R11); Go core with CLI/TUI/REST (R3 core); full CLI-agent integration via plugin + MCP + ACP services (R4); learn-from-real-codebase experience pipeline (record → triage → process → universal knowledge) (R2); deep research incl. CodeGraph under-the-hood + memory systems (done); vector DB + Postgres; Docker/Podman Compose via **systemctl user-space** + `scripts/{start,stop,restart,status,install,...}`; docs to nano-detail w/ diagrams+schemes+graphs, all SQL/template definitions; final **zip** + **tar.gz** packaging retrievable to filesystem.

PINS / CORRECTS (update the plan): - **Wire format** = **TOON, JSON fallback (NOT TOML).** Original: "Toon instead of JSON with JSON capability as fallback." MVP SPEC §2/§6 + the committed `api/openapi.yaml` used JSON+TOML → API serialization must become **TOON primary** + **JSON fallback**; openapi.yaml content-negotiation REVISION QUEUED. `config.toml` stays TOML. DEFAULT (override-able): on-disk skill files stay TOML+Markdown (human-editable, git-versionable R14). → **TOON is the REAL format `github.com/toon-format/toon`** (operator-confirmed via a direct repo link; `git ls-remote` verified reachable — default branch `main`, HEAD a19a117). The blueprint TXT's "interpret Toon as TOML" is **SUPERSEDED** — we implement/vendor a **Go TOON codec** and serve `application/toon` (primary) + `application/json` (fallback). Queued: TOON serialization layer + openapi.yaml revision. - Transport: **Quic/HTTP3** + **Brotli** (aligns; supersedes MVP HTTP2-default). - **ACP** (Agent Client Protocol) confirmed alongside MCP + plugins. - **Endless, deeply-recursive** skill branching: every technology appearing in any dependency chain gets its own Skill, recursively, without bound. - Explicit deliverable: a **feasibility** + **step-by-step methodology** + **danger-zones** document ("Can we do this? Here's exactly how"), backed by deep web research. - Skills must be directly usable by Claude Code / OpenCode / other CLI agents.

Requirement clusters (from operator, in order received)

- **R1 — Standard.** Build+run clean; 4-layer test coverage + paired mutations (Constitution §1/§1.1/§11.4); fix flagged security defects (CORS reflect+credentials; api_key in query/log); dedupe `project/` vs `skill-system/` to one canonical tree.
- **R2 — Universal** + **dynamic.** Not Android-specific; create skills on demand for any technology set. Mentioned tech (Gin, quic-go,

pgvector, tree-sitter, mcp-go, Bubble Tea, Cobra, brotli, TOML) must be **working POCs**, not stubs.

- **R3 — Clients.** CLI, TUI, REST API, **Web**, **Desktop (Win/macOS/Linux)**, **Mobile: Android, iOS, HarmonyOS, Aurora OS (auroraos.ru)**. **Maximize shared codebase** across all surfaces. (Web/Desktop/Mobile are greenfield.)
- **R4 — Agent interop.** Usable from Claude Code (toolkit + aliases), OpenCode, Kimi Code, and **HelixTrack** / **HelixAgent** / **HelixLLM** (`../helix_code/`).
- **R5 — Incorporation analysis.** In-depth research on how to integrate all of R4 properly (→ 4 research agents in flight).
- **R6 — Canonical use case (wizard).** User opens a client → **skill-creation wizard** → enters a tech set (example: `android, android-aosp, java, kotlin, c++, cmake`) → submit → backend runs **create** → **map (DAG)** → **full processing** (generate content, resolve deps, validate via jury, embed, store) → progress reported back to the client.
- **R7 — Model access.** Obtain quality models via **LLMsVerifier**, **HelixLLM**, **Claude Toolkit aliases** — pluggable ModelProvider, not hardcoded OpenAI. Any required dependency not present in the parent dir must be **vendored as a submodule at `submodules/<snake_case_name>/`** per Constitution naming.
- **R8 — Exhaustive testing.** Every unit covered by ALL supported test types PLUS **Challenges** and **HelixQA test banks/suites**; coverage as close to **100%** as possible. (Helix ships `helix_qa` + `challenges` submodules and a `helixqa` binary — these are real components, not abstractions. Exact rules ← research agent 5.)
- **R9 — Submodule resolution + sync.** All deps and dependency submodules live under `submodules/<snake_case>/`, OR reuse the versions from the parent dir/submodules — **parent-dir versions have PRIORITY**. BOTH copies must always be in sync with main/master. (Exact Constitution rule ← research agent 5.)
- **R10 — Docs Chain.** Fully incorporate the **Docs Chain** submodule (`.docs_chain`) per Constitution. (Rules ← research agent 5.)
- **R11 — Zero bluff, anywhere.** No false results, faulty results, or faulty codebase; no bluff of any kind or form. Positive-evidence-only everywhere. This governs every gate, test, and status report (Constitution §7.1/§11.4/§1.1).
- **R12 — OpenDesign for all design.** Every client's design/styling/diagrams/ illustrations MUST use **OpenDesign** (`git@github.com:nexu-io/open-design.git`). Deliver all Constitution-mandated design artifacts: wireframes, sketches, Figma, and exports in PDF, PSD, SVG (+ other Constitution-defined types). [Incorporation research: NEXT WAVE.]

- **R13 — Validation skill corpus.** The system must successfully create + validate this corpus (this IS the end-to-end proof): android, android-aosp, rockchip, orange-pi, java, c, c++, kotlin, python, postgres, bash, go, gin-gonic, flutter, angular, typescript, javascript, major web/mobile/desktop frameworks (all OSes), linux, macos, debugging, cmake, make, gcc, bazel, brotli, quic, http3, http, protocols, design-patterns, algorithms, security, snyk, sonarqube, maven, gradle.
- **R14 — Git-versioned + real-time growth.** Every created/obtained skill is **Git-versionable** — persisted as versioned files so the main repo grows as knowledge expands (TOML per SPEC §4.2, committed; kept in sync with the Postgres+pgvector index via import/export). **MCP / ACP / plugins** must trigger real-time skill creation + deep research, expanding/updating the knowledge base and skill graph live, improvements available immediately.

Architecture notes forced by R13/R14

- Dual persistence: skills live as **versioned TOML files in-repo** (source of truth for git history) AND in **Postgres+pgvector** (query/search index); a sync path keeps them consistent (SPEC §6 import/export is the seam).
- The MCP server is not just read: it exposes **create/expand/learn** tools that kick off (async) generation + research jobs and stream progress — the R6 wizard and R14 real-time growth run through the same job pipeline.

Baseline (captured evidence, 2026-07-15)

- Go 1.26.4 present. `go.mod` = `github.com/helixdevelopment/skill-system`, 18 deps, no `go.sum` (never resolved). 53 `.go` files, **0 tests**.
- `go build ./...` → **FAILS**: duplicate `color*` consts (common.go vs skill.go); unused imports (`strconv`, `context`); undefined `pgvector.RegisterTypes`, `stdlib.ConnPool`, `stdlib.GetPool`; `tls.Config.Allow0RTT` unknown field.
- `go mod tidy` pulled **five competing ORMs** (ent, gorm, uptrace/bun, go-pg, sqlx) alongside pgx — incoherent generated skeleton.
- Security: CORS reflect-origin + `Allow-Credentials: true` (HIGH); api_key via query param then logged (MEDIUM). Present in both duplicated copies.

> **SUPERSEDED — this Baseline is the ORIGINAL as-found snapshot, not the current state.** The `go build ./...` **FAILS** finding above was resolved by the P0 build-fix (commit `5532e2b`, "Make Go backend build+vet green") — the backend now compiles clean (`go build ./...`=0; single pgx DB layer, the five competing ORMs purged, `go.sum` committed). The CORS + api_key security findings were resolved by the G01 runtime-security fix (commit `1a1a3f3`, single hardened listener) — see the GAPS_AND_RISKS_REGISTER G01 STATUS block. This block is retained as the §11.4.7 regression oracle (the known-bad baseline every fix is diffed against); read it as HISTORY, never as the present tree.

Global constraints (Constitution)

No bluffing / positive-evidence only; every gate paired with a mutation; no guessing language; credentials never committed (templates only); multi-upstream push; deep understanding + captured evidence before implementation; stop-and-fix at root cause on any discovered defect.

Open architecture decisions (pending research → operator sign-off)

1. **Shared-core tech** for CLI/TUI/Web/Desktop/Mobile(incl. Aurora/Harmony) — maximize reuse (research agent 3).
2. **Model-access abstraction** — LLMsVerifier/HelixLLM/alias backends (agent 4).
3. **Agent-interop layer** — one MCP server + CLI + aliases vs bespoke plugins (agent 2).
4. **Helix ecosystem integration** — HelixTrack/HelixAgent/HelixLLM surfaces (agent 1).
5. **Canonical location** for the deduped service tree (top-level `skill-system/` vs under this research folder).

Research findings — CONFIRMED (agents 1-3 of 5, cited)

- **Interop is solved by MCP.** Claude Code, OpenCode, and Kimi Code all speak MCP natively over stdio. Ship **ONE stdio MCP server + a thin CLI + shell aliases** — no bespoke plugins. Register once in

claude_toolkit's shared plugins (`~/.claude-shared/plugins/`), add a sibling entry in `helix_code/opencode.jsonc` (already has a `codegraph` MCP entry), and `mcpServers` for Kimi. Optional `SKILL.md`.
- **Models (R7, partial).** HelixAgent (`:7061`) and HelixLLM (`:8443`) are both **OpenAI-compatible** → one provider-agnostic client, swap base URL; use `/v1/embeddings` (768d) + `/v1/chat/completions`; `llms-verifier`/`helixqa` are PATH binaries. HelixAgent/HelixLLM are submodules under `helix_code/submodules/`. (LLMsVerifier specifics + ModelProvider abstraction ← agent 4, running.)
- **Helix ecosystem.** HelixTrack = Go/Gin REST `:8080` JWT **sibling** repo → integrate via REST + register our MCP server in `helix_track/.mcp.json`. Plug-in idiom for owned components: submodule + Makefile `build` + PATH binary + `.mcp.json` entry + `internal/<name>/` REST adapter.
- **Shared-core = contract-first thin clients.** Every surface is a thin client over the REST/MCP API; genuine shared logic is only ~15-30%. Make **OpenAPI + the MCP schema the single source of truth**, codegen clients, enforce with contract tests. Per surface: **CLI+TUI = Go** (reuse existing); **GUI apps = Flutter** — the ONLY framework covering Android + iOS + **HarmonyOS** (Flutter-OHOS) + **Aurora OS** (omprussia embedder) + desktop from one codebase; **Web = React**. Build order: CLI → Web → Desktop → Android/iOS → TUI → HarmonyOS → **Aurora last** (highest risk).
- Still running: agent 4 (model access), agent 5 (Constitution: test taxonomy / submodule sync / Docs Chain).

Draft staged plan (to be finalized after research; foundation-first)

- **S0 Research** (in flight): 4 agents — Helix ecosystem, agent interop, shared-core, model access.
- **S1 Foundation**: make backend compile & run; pick ONE db layer (pgx); remove the 4 stray ORMs; fix security defects; dedupe to one tree; `go build ./...` + `go vet` green; docker-compose up (Postgres+pgvector) smoke test.
- **S2 Core correctness + tests**: 4-layer tests (unit/integration/e2e/contract)
  - paired mutations per Constitution; DAG logic, recursive CTE, search proven.
- **S3 Model + processing**: ModelProvider abstraction; wizard pipeline (create→map→validate→embed→store) end-to-end (R6).
- **S4 Interop**: MCP server + CLI + aliases; Helix* integration (R4).

- **S5 Shared core + clients**: shared-core lib → CLI/TUI/REST, then Web, Desktop, Mobile (Android/iOS first; Harmony/Aurora per research risk).
- **S6 Hardening/docs/packaging**: security, ops, docs to nano-detail, packaging.

Each stage lands only with captured build/test evidence; SDD dispatches per task.

Addenda — 2026-07-15 (new operator mandates, folded in)

- **TOON correction (serialization):** supersedes blueprint "Toon = TOML". Real format = `github.com/toon-format/toon` (ls-remote OK, branch main, HEAD a19a117). API wire = `application/toon` primary + `application/json` fallback; needs a Go TOON codec. openapi.yaml content-negotiation revision QUEUED.
- **R15 — systemctl (user scope) + ops scripts:** system MUST integrate via `systemctl --user` with a user-scope unit, plus `scripts/` containing at least start, stop, restart, status, install (and uninstall/logs). Docker/Podman Compose (pgvector/pgvector:pg16) orchestrated underneath. Matches blueprint deploy section; now an explicit hard requirement. **Stream B dispatched.**
- **R16 — skill granularity & composition:** big technologies MUST be decomposable into smaller atomic skills wrapped by an umbrella/composite skill, with a first-class relationship mechanism (typed edges: composes/part_of, depends_on, requires, extends, prerequisite, related_to, alternative_to) usable as building blocks for whole stacks. Research + data-model design **Stream A dispatched** (research/skill_granularity_and_composition.md).
- **R17 — exhaustive gaps/risks remediation + total test coverage:** ALL gaps, weak spots, inconsistencies, danger zones, and potential/existing issues MUST be in-depth researched, documented, and given a proper tackle-decision; every covered point MUST be heavily covered with all supported test types + Challenges + HelixQA test banks and fully validated/verified/confirmed as complete success — no false or faulty results, no bluff of any kind, anywhere. **Deliverable landed:** GAPS_AND_RISKS_REGISTER.md (27 findings G01–G27, 4 CRITICAL) + research/testing_infrastructure_plan.md (per-gap coverage matrix, all 13 test types). P0.5 remediation spine in progress (G01 CLOSED, Fable-xhigh GO).

- **R18 — full documentation delivery, always in sync:** the whole project MUST ship with complete documentation — API docs (static AND interactive), user manuals, guides, tutorials, FAQs — plus ALL diagrams / schemes / graphs with stunning OpenDesign illustrations (R12 / §11.4.162 / §11.4.190), and all SQL definitions, templates, and other materials. Everything MUST be exported to every mandatory file type per constitution (§11.4.65) and be ALWAYS up to date and in sync, wired through the required hooks + the Docs Chain submodule (R10 / §11.4.106) so nothing ever drifts. Composes R10 + R12 + §11.4.12/.44/.45/.53/.56/.57/.59/.60/.65/.106/.168/.170/.190. Architecture design **stream dispatched** (research/r18_documentation_delivery_design.md).
- **R19 — Anthropic API support (first-class ModelProvider + interop):** besides full OpenAPI-contract compatibility (the system's own REST surface, G09), the system MUST support Anthropic's API(s). Primary reading (composes R7 pluggable ModelProvider + R4 interop): the ModelProvider/`LLMClient` layer MUST support Anthropic's **Messages API** as a first-class provider — usable for the LLM jury (G05) and auto-growth generation (G20) — decoupled from any concrete `*OpenAILLM` (G20 already removes that coupling + flags the missing `NewLLMClientFromConfig` factory, the R19 plug-in point). Anthropic offers no first-party embeddings API — embeddings stay on the G10 provider set (local / OpenAI-compatible / Voyage), documented explicitly, never faked. Secondary facet (flagged, design resolves): whether to ALSO expose an Anthropic-Messages-compatible surface for R4 Claude-Code/agent interop — surface any genuine sub-decision per §11.4.66. Credentials via the §11.4.10 single-source (field names only, never values). Design **stream dispatched** (research/r19_anthropic_api_support_design.md).
- **R20 — Containers submodule for ALL containerization (operator mandate 2026-07-15):** every containerized workload (dev/test infra, build sandboxes, deploy, HelixQA infra boot) MUST go through the vasic-digital Containers submodule (`git@github.com:vasic-digital/containers.git`, existence verified via gh) — NO ad-hoc docker/podman outside its `pkg/boot`/`pkg/compose`/`pkg/health` layer (inherits §11.4.76 + rootless §11.4.161). The ops-hardening design (G13 compose profiles) MUST route through it; vendoring is gated on the G14/X1 submodule-policy decision. Folded into the G14/X1 decision package (research/g14_x1_submodule_policy_decision.md).
- **R21 — adopt reusable Helix-family practices (operator mandate 2026-07-15):** survey the sibling `helix_*` projects under the shared parent projects dir for reusable universal architecture /

submodules / practices this project MUST follow — especially Containers (R20), HelixQA integration (R8/§11.4.27), Docs Chain (R10/§11.4.106), and other major sub-systems / sub-modules (§11.4.74 catalogue-first extend-don't-reimplement, §11.4.28 equal-codebase). Survey **stream dispatched** (research/helix_family_reusable_practices.md).

- **R22 — catalogue-first incorporation from vasic-digital + HelixDevelopment orgs (operator mandate 2026-07-15):** INCORPORATE the universal reusable submodules these two orgs already provide rather than re-implement (§11.4.74 extend-don't-reimplement; §11.4.28 equal-codebase; §11.4.31 recursive helix-deps). gh-enumerated high-value matches: **LLMProvider** (`vasic-digital/LLMProvider` — shared LLM provider interface, 40+ adapters incl. Anthropic, retry/circuit-breaker/health → supersedes the hand-rolled `OpenAILLM` + planned R19 `AnthropicLLM`; reshapes R7/R19), **MCP_Module** (`vasic-digital/MCP_Module` → internal/mcp, R14), **http3** (`vasic-digital/http3` drop-in HTTP/3 → replaces direct quic-go), **docs_chain** (R10/R18), **HelixQA** (`HelixDevelopment/helixqa`, R8), **containers** (R20), **design_system** + **open-design** (R12), **DebateOrchestrator** (`HelixDevelopment/DebateOrchestrator` multi-agent debate → the G05 jury), **DagOrchestrator** (G11 worker scheduling), **PipelineRuntime** (validation pipeline), **token_optimizer** (§11.4.141), **VisionEngine**/**Panoptic** (client UI visual proof §11.4.170), the **KMP** component suite (R3 clients). When a needed capability is missing, EXTEND/IMPROVE the upstream submodule (never fork-and-diverge); include ALL transitive dependency submodules each brings (§11.4.31 recursive). Incorporation plan folded into the Helix-family survey + a follow-on full-catalogue pass; all incorporation gated on the G14/X1 submodule-policy decision.
- **R23 — full constitutional compliance, no violations, no bluff (operator mandate 2026-07-15):** every rule / mandatory constraint / guideline / technology / system derived from the constitution submodule MUST be fully followed, respected, and applied with NO violation, ignorance, or skipping; ALL of it MUST be processed once the project is fully implemented, with no violation of any kind confirmed to exist, and no bluff of any form anywhere. Operationalised as a standing constitutional-compliance gate: (a) a compliance-ledger audit enumerating every BINDING anchor (§11.4.x/§12.x/§1–§10) vs current project state → evidence-backed COMPLIANT / VIOLATION / PENDING-AT-COMPLETION / N-A (`research/constitution_compliance_audit.md`, **stream dispatched**); (b) wire the `constitution/scripts/gates/` CM-gates + a §11.4.32 project sweep; (c) a full re-run at project completion so

zero violations is PROVEN, never assumed (§11.4.6). Composes §11.4.17 / §11.4.32 / §11.4.75 / §11.4.201 / §11.4.209 + the whole §11.4 anti-bluff covenant.
- **R24 — every operator request always respected + recorded, zero request-loss (operator mandate 2026-07-15):** every request/prompt the operator issues MUST ALWAYS be respected, taken into account, executed, and processed — NO avoiding, ignoring, skipping, or any form of loss. Operationalised via the §11.4.208 project-local operator-request ledger (`requests/history.md`, **created**): every request recorded newest-first with content + accepted-timestamp(TZ) + Track + alias + model+effort + processing Disposition, reconciled against this REQUIREMENTS.md SoT so an acted-on-but-unrecorded request is caught. **Intake audit performed:** R1–R22 were already SoT-tracked; R23 + R24 were acted-on-but-not-recorded → recorded this turn. Follow-up **G38** tracks wiring the §11.4.208(D) auto-capture hook (a `UserPromptSubmit`-class hook appending a row per new prompt) so future intake is captured at accept-time, not reconstructed. Composes §11.4.197 (loss-of-requirements FORBIDDEN) + §11.4.208 + §11.4.202 + §11.4.6.
- **R25 — canonical project key `hxs`, used EVERYWHERE (operator mandate 2026-07-15):** the project name is HelixSkill; the canonical project key MUST be **`hxs`** (lowercase, §11.4.29). It MUST be used everywhere — as the ticket/workable-item id prefix (`hxs-NNN`, superseding the G0x/R0x/P0x scheme) AND as the §11.4.151 release/version prefix (`hxs-<version>`). ALL occurrences MUST be fully updated and ALL references too, everywhere — with ZERO broken references (§11.4.6: a rename is forbidden without proving every reference resolved). Executed as a DEDICATED integrity-gated pass (deterministic id-map G0x/R0x/P0x → hxs-NNN; a post-rename gate asserting zero orphan old-ids + every hxs-NNN resolvable + no broken links), run once the current design-doc churn settles so the FULL doc set renames atomically. Composes §11.4.29/§11.4.54/§11.4.151/§11.4.6/§11.4.124.

Operator decisions RESOLVED (2026-07-15) — supersede the pending sign-off items above

- **G14/X1 submodule policy → VENDOR FRESH under this project.** Operator chose: each dependency (LLMProvider, http3, docs_chain, containers, dag_orchestrator, PipelineRuntime, design_system,

open-design) is vendored FRESH as a git submodule under THIS project's own `submodules/<snake_case_name>/` (§11.4.28(C) `<project_root>/submodules/`), independent of `helix_code`. Supersedes the autonomously-adopted single-canonical consume-by-reference (Option A). The R22-catalogue ADOPT verdicts stand; the LAYOUT becomes project-local fresh vendoring + `install_upstreams` (§11.4.36) + recursive helix-deps (§11.4.31). open-design dir → `submodules/open_design/` (§11.4.29 snake_case, our own tree). Vendoring lane UNBLOCKED.

- **§11.4.185 manual QA → QA team at milestones.** Drive to autonomous-GREEN + captured evidence + build, then hand off to the operator's QA team for the FINAL manual sign-off; completion/release tags wait on that confirmation. The §11.4.126 release-terminal precondition for this project.

- **R7/R19 credentials → `~/api_keys.sh` OR local `.env` (both supported, like other Helix projects).** The provider layer loads API tokens from `$HOME/api_keys.sh` and/or a local `.env` (both MUST be supported). §11.4.10 single-source: field NAMES only ever in git, values never; `.env` stays gitignored (§11.4.30). Unblocks R7/R19 live acceptance.

- **Security:** sync_submodules.sh hardened (fail-closed validation; paired attack proof) — committed c473d01.

- **Seed corpus:** R13 validation corpus + 8 real seed skills committed 0e0bc3b (43 nodes / 56 edges, DAG acyclic, independently re-verified).